Coding Patterns

Study Guide

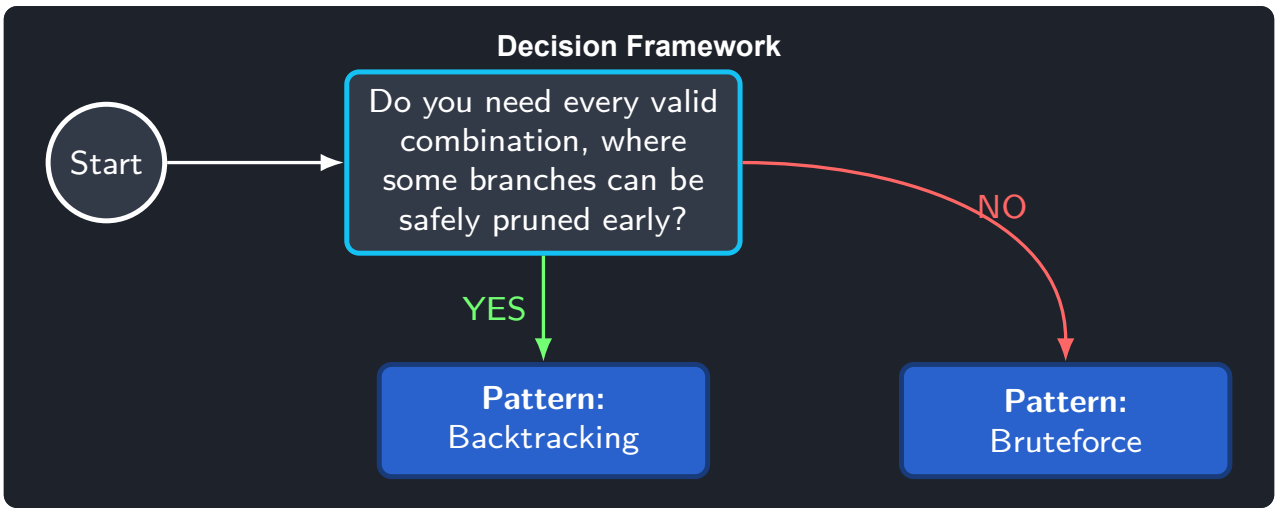
A fast-reference book of algorithmic patterns, decision frameworks,
tips and Python implementations

Vlamaz

September 7, 2026

Table of Contents

Chapter 1
Backtracking & Bruteforce



1.1 Backtracking

Green Flags -- When to Use Backtracking

- We need to generate all the possible permutations of some symbols, where we can cut some branches of exploration.

Tips

The branching factor of this search is bounded by the n-th Catalan number,

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \Theta\left(\frac{4^n}{n^{3/2}}\right),$$

which counts exactly the number of valid balanced-bracket sequences of length $2n$. That is why the pattern template's complexity is written as $O(n \cdot 4^n/\sqrt{n})$: each of the $\Theta(4^n/\sqrt{n})$ valid sequences costs $O(n)$ to materialize.
In practice you rarely need to derive the bound – what matters is recognizing the shape: every partial candidate is a state carrying the value built so far, plus whatever counters describe *how many more moves are still legal* (e.g. `opening_used`, `closing_used`). A state is only ever pushed back onto the exploration queue if it is still guaranteed to be completable into a valid answer – that early rejection of dead branches is exactly what separates backtracking from plain bruteforce.

Approach

Model the problem as growing a partial answer one decision at a time, taking a decision only if the partial answer can still be completed into something valid. This constant validity check keeps the explored space close to the number of *valid* outcomes rather than *all* outcomes, which is what makes backtracking dramatically faster than bruteforce despite sharing the same explore-and-expand skeleton.

Pattern Template

```python
"""
Pattern: backtracking (state-space search with pruning)

Time: O(n * 4^n / sqrt(n))
Memory: O(n * 4^n / sqrt(n))

n * 4^n / sqrt(n) bounds the n-th Catalan number: the number of valid
bracket combinations of length 2n, which is exactly the branch count
this pattern explores.
"""

from collections import deque
from dataclasses import dataclass


@dataclass
class State:
    prefix: str
    opening: int
    closing: int


def generate_bracket_combinations(n: int) -> list[str]:
    # ================================================================================
    # Stage 1: Seed the queue with the empty state (no brackets placed yet)
    # ================================================================================
    queue: deque[State] = deque([State("", 0, 0)])

    # ================================================================================
    # Stage 2: Expand every state until all valid strings are complete
    # ================================================================================
    while ...:
        # ============================================================================
        # Stage 3: Pop the state currently being expanded
        # ============================================================================
        current: State = ...

        # ============================================================================
        # Stage 4: Only re-queue states that stay completable, so an invalid
        # combination is never built. Ask: can I still add "("? Can I still
        # add ")"? This pruning is what turns bruteforce into backtracking.
        # ============================================================================
        ...

    return [state.prefix for state in queue]
```

Worked Example

```python
"""
Given number n, you need to generate all the possible combinations of same pair symbosl ().
It should have the opening and closing symbols in the same pair.
The idea of the optimal solution is to generate all the good combinations from start

Example:
```

```
n = 3
result ["((()))", "(()())", "(())()", "()()()"]
"""


def generate_symbol_pair_combinations(n: int) -> list[str]:
    """
    The main idea here is to use counters while we are generating the combinations.

    In this approach we wil have the running queue with a stete of 3, the value of stroken,
    the opening and closing symbol counters.
    """
    result: list[str] = []

    # To make the code more functional, we will do the loop approach, so we need to use the queue in order to have the
        things to process
    running_queue: list[tuple[str, int, int]] = [("", 0, 0)]

    while running_queue and len(running_queue[0][0]) < 2 * n:
        # As in bruteforce, we need to extract the first element from the queue
        first_element, opening, closing = running_queue.pop(0)

        # First we will be adding the opening symbol, then the closing one
        if opening < n:
            new_element: tuple[str, int, int] = (
                first_element + "(",
                opening + 1,
                closing,
            )
            running_queue.append(new_element)

        if opening > closing:
            new_element: tuple[str, int, int] = (
                first_element + ")",
                opening,
                closing + 1,
            )
            running_queue.append(new_element)

    return [prefix for prefix, _, _ in running_queue]


if __name__ == "__main__":
    print("Generating symbol pair combinations for 3:")
    res = generate_symbol_pair_combinations(3)
    print(res)
```

1.2 Bruteforce

Green Flags -- When to Use Bruteforce

- We need to find all the possible combinations between things, including sublists or interchanges of elements without any prohibitions.
- The order inside the combinations doesn't matter.

Tips

This variant enumerates every combination with no pruning at all: each of the n digits contributes up to 4 candidate letters, so the search tree has branching factor 4 and depth n, producing $\Theta(4^n)$ leaves. Because every partially-built combination is also kept on the queue while it grows, both the running time and the memory bound land on $O(n \cdot 4^n)$.
The state carried in the queue can be as small as the string built so far – there is no extra validity

counter needed, because unlike backtracking there is no early-rejection check that would let a branch be discarded. If you find yourself adding a condition of the form "this partial combination can never be valid, so skip it," you have rediscovered backtracking and should switch pattern.

Approach

Generate every possible combination with no attempt to discard invalid or unpromising branches early – correctness comes purely from exhaustiveness. Track just enough state (the partial result plus a cursor into the source data) to avoid producing the same combination twice, and accept that the running time will be exponential in the input size.

Pattern Template

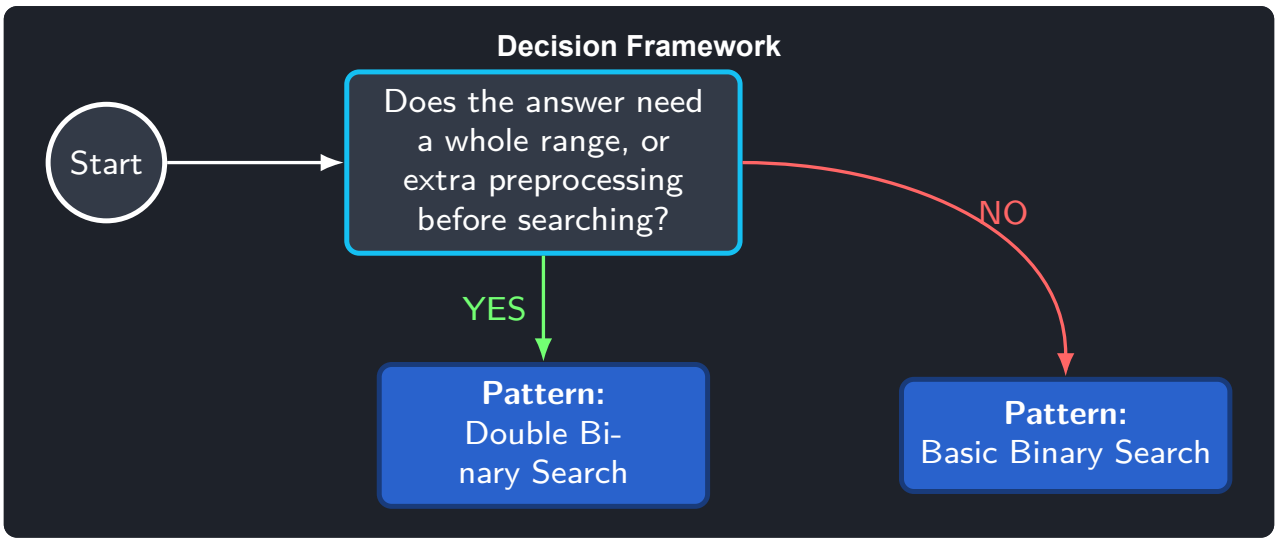
```
"""
Pattern: bruteforce combination generation

Every digit maps to up to 4 letters, so the search tree branches by
at most 4 at each of the n digits, with no pruning of any branch.

Time: O(n * 4^n)
Memory: O(n * 4^n)
"""

from collections import deque

DIGIT_TO_LETTERS = {
    "2": "abc", "3": "def", "4": "ghi", "5": "jkl",
    "6": "mno", "7": "pqrs", "8": "tuv", "9": "wxyz",
}


def generate_combinations(digits: str) -> list[str]:
    # ================================================================================
    # Stage 1: Start the queue with a single empty combination
    # ================================================================================
    queue: deque[str] = deque([""])

    # ================================================================================
    # Stage 2: Keep expanding until every combination has one letter per digit
    # ================================================================================
    while ...:
        # ============================================================================
        # Stage 3: Take the shortest unfinished combination out of the queue
        # ============================================================================
        current: str = ...

        # ============================================================================
        # Stage 4: Try every letter for the next digit and push the longer
        # combinations back -- unlike backtracking, nothing is pruned here.
        # ============================================================================
        ...

    return list(queue)
```

Worked Example

```
"""
* Given a list of a unique numbers nums. We should find all the possible combinations between them,
* including the basic list and the nums. The order inside the combinations doesn't matter.
* Same combinations should not repeat.
*
```

```
    * e.g.
    * nums = [3,6,17]
    *
    * output = [[], [3], [6], [3,6], [17], [3,17], [6,17], [3,6,17]]
    *
    * Time: O(n*2^n)
    * Memory: O(n*2^n)
    """


def get_all_not_repeated_combinations(nums: list[int]) -> list[list[int]]:
    if len(nums) <= 0:
        return []

    result: list[list[int]] = [[]]
    # We need this list in order to have the running state of the combination and know when to end the loop
    running_state: list[tuple[list[int], int]] = [([], 0)]

    while running_state and len(running_state[0][0]) < len(nums):
        # Now we should obtain the prefix from the front and drop it
        prefix: list[int] = running_state.pop(0)

        # Once we have the next prefix, we should iterate over the nums in the
        # original list and add the new combinations to the running state so we can combine all
        for i in range(prefix[1], len(nums)):
            # Creating the new prefix combination
            new_prefix: list[int] = prefix[0] + [nums[i]]

            # Now we should add the combinations to the result
            result.append(new_prefix)

            # And finally add the new combination as a State to the state list
            running_state.append((new_prefix, i + 1))

    return result


if __name__ == "__main__":
    print("Combinations for: 3 6 17:")
    nums: list[int] = [3, 6, 17]
    res: list[list[int]] = get_all_not_repeated_combinations(nums)

    print(res)
```

Chapter 2
Binary Search



Assumes the search space can first be split into a clean `good' prefix/suffix and a bad' one – if it cannot, binary search does not apply at all.

2.1 Basic Binary Search

Complexity

Time: O(log(n)) **Memory:** O(1)

Green Flags -- When to Use Basic Binary Search

- If we can split the elements into two lists, where first list is the good elements and the second is the bad ones.

Approach

Binary search only works once the search space can be split into two contiguous zones: everything satisfying some monotonic predicate (`good`) and everything that does not (`bad`). Each iteration evaluates the middle element, discards the half that cannot contain the boundary between the two zones, and repeats until the boundary itself is found – the real design work is deciding, for the concrete problem, what `good` means and which pointer should end up holding the answer.

Pattern Template

```python
"""Pattern for the basic binary search

Time: O(log(n))
Memory: O(1)
"""


def search_last(nums: list[int], target: int) -> int:
    # ============================================================================
    # Stage 1: Initialization
    # 1) Looking for last good, then response in pointer left -> l=0, r=len(nums)
    # e.g. Search for the last index on the list that x
    # 2) Looking for first bad, then response in pointer right -> l=-1, r = len(nums) -1
    # e.g. Search for the first index on the list that is not x
    # ============================================================================
    left: int = 0
    right: int = len(nums) - 1

    # ============================================================================
    # Stage 2: Main loop, use this instead of left < right so it can handle better the case
    # ============================================================================
    while right - left > 1:
        # ============================================================================
        # Stage 3: Find the middle, this is the int overflow approach, in Python etc,
        # (right + left) // 2 is good enough
        # ============================================================================
        middle: int = left + (right - left) // 2

        # ============================================================================
        # Stage 4: Decrementing the search space
        # ============================================================================
        if (
            ...
        ): # MAIN PROBLEM HERE; WHICH WILL SPLIT THE RESPONSES IN GOOD AND BAD ONES
            left = middle
        else:
            right = middle
    # ============================================================================
    # Stage 5: Response processing
    # ============================================================================
    ...
```

Worked Example

```python
"""
Given a sorted array of numbers nums and a number tg.
We should find the last index of the number tg in the list.

e.g.
nums = [1,2,6,6,9,10,11,13,14]
tg = 6
output = 3
"""


def find_last_index_for_n(nums: list[int], target: int) -> int:
    # ============================================================================
    # 1) Looking for last good, then response in pointer left -> l=0, r=len(nums)
    # 2) Looking for first bad, then response in pointer right -> l=-1, r = len(nums) -1
    # ============================================================================
    left, right = 0, len(nums)

    while (
        right - left > 1
    ): # We use this instead of left < right so if they are equal, they don't go into infinite loop
```

```
        middle: int = left + (right - left) // 2

        # ============================================================================
        # Typical problem: use left and right inside the good function.
        # Inside this function we should only use the middle value.
        # ¡Again, do not f. use the left and right in here!
        # ============================================================================
        if nums[middle] <= target:
            left = middle
        else:
            right = middle

    return left if nums[left] == target else -1


if __name__ == "__main__":
    nums: list[int] = [1, 2, 6, 6, 9, 10, 11, 13, 14]
    target: int = 14

    last_index: int = find_last_index_for_n(nums, target)

    print(f"Last index for {target} in {nums}: {last_index}")
```

2.2 Double Binary Search

Complexity

Time: O(2*log(n)) -> O(log(n)) **Memory:** O(1)

Green Flags -- When to Use Double Binary Search

- We need to find some range
- we need to run one preprocessing binary search, in order to prepare the ground for the main binary search.

Approach

Some problems need more than a single index as the answer – a whole range, or a value that first needs its own preprocessing step. Run one binary search to establish that preprocessing (or the first boundary), then a second, independent binary search on top of it to pin down the remaining boundary; the two searches are usually near-identical in shape but test a different predicate.

Pattern Template

```
"""
Pattern double binary search:
The idea is to use two binary searches, one to search for the first elements of the
list be looking for the target in the bad side, and the other binary search
looking for the last element of the target in the good side.

Time: O(2*log(n)) -> O(log(n))
Memory: O(1)
"""


# ============================================================================
# Stage 1: Searching for the first element of the target in the list
# ============================================================================
def search_first(nums: list[int], target: int) -> int:
```

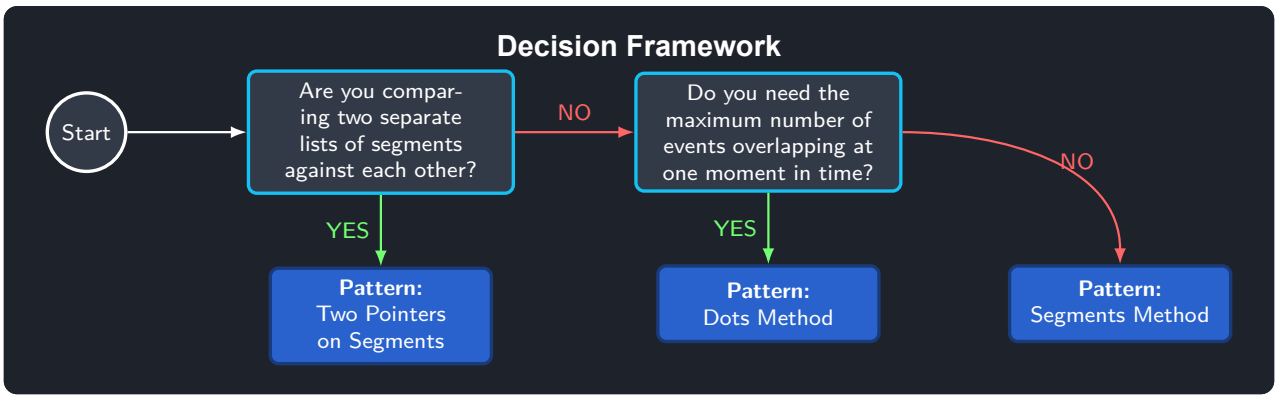
```
    # This is the approach to search for the bad element of target, the right part
    left, right = -1, len(nums) - 1

    while right - left > 1:
        middle: int = left + (right - left) // 2

        # The base problem here is to undertand which is the good
        # Function will be used here
        if ...:
            left = middle
        else:
            right = middle
    return ...


# ================================================================================
# Stage 2: Searching for the last element of the target in the list
# ================================================================================
def search_last(nums: list[int], target: int) -> int:
    # This is the approach to search for the good element of target, the left part
    left, right = 0, len(nums)
    while right - left > 1:
        middle: int = left + (right - left) // 2

        # The base problem here is to undertand which is the good
        # Function will be used here
        if ...:
            right = middle
        else:
            left = middle
    return ...


# ================================================================================
# Stage 3: Processing the results of the both searches
# ================================================================================
def search_range(nums: list[int], target: int) -> list[int]:
    # Processing the results of the two searches.
    ...
```

Worked Example

```
"""
Given a sorted array of numbers nums and a number tg.
We should find the first and the last index of the number tg in the list.

e.g.
nums = [2,3,3,3,3,7,8]
tg = 3
response = [1,4]
"""


def search_first(nums: list[int], target: int) -> int:
    """Using the first binary search to look for the bad element, the first changed."""
    left: int = -1
    right: int = len(nums) - 1

    while right - left > 1:
        middle: int = left + (right - left) // 2

        if nums[middle] < target: # Should be only equal to excludethe left
            left = middle
        else:
            right = middle
    return right if nums[right] == target else -1

```

```python

def search_last(nums: list[int], target: int) -> int:
    """Using the binary search to look for the first good element, the left part basically."""
    left: int = 0
    right: int = len(nums)
    while right - left > 1:
        middle: int = left + (right - left) // 2
        if nums[middle] <= target:
            left = middle
        else:
            right = middle
    return left if nums[left] == target else -1


def find_first_and_last_index(nums: list[int], target: int) -> list[int]:
    """Using the double binary search. Processing the binary search results."""

    if not nums:
        return [-1, -1]

    return [search_first(nums, target), search_last(nums, target)]


if __name__ == "__main__":
    nums: list[int] = [2, 3, 3, 3, 3, 7, 8]
    target: int = 3

    print(
        f"First and last index for {target} in {nums}: {find_first_and_last_index(nums, target)}"
    )
```

Chapter 3
Dots and Segments



3.1 Dots Method

Complexity

Time: O(n*log(n)) **Memory:** O(n)

Green Flags -- When to Use Dots Method

- You must fint the maximum number of concurrent meeting or something related to times.

Approach

Treat every interval as two events – a start and an end – flatten the events from every interval into one sorted timeline, and sweep across it while keeping a running counter of how many intervals are currently open. The counter's maximum value across the sweep answers "how many intervals overlap at the busiest moment", without ever comparing intervals pairwise.

Pattern Template

```
"""
Pattern: Dots method

Time: O(n*log(n))
Memory: O(n)
"""
```

```python
def minimum_meeting_rooms(segments: list[list[int]]) -> int:
    # ================================================================================
    # Stage 1: We transform the segments into points
    # The start is represented as +1 and the end as -1
    # ================================================================================
    points: list[list[int]] = []
    for segment in segments:
        ...

    # ================================================================================
    # Stage 2: Sort the points by time
    # ================================================================================
    points.sort()

    # ================================================================================
    # Stage 3: find the solution. this is the main problem, how to form the good response
    # ================================================================================
    max_rooms: int = 0
    curr_room: int = 0

    for point in points:
        ...

    return max_rooms
```

Worked Example

```python
"""
Given a list of segments where each segment represents the time of a meeting in a cabine.
Each segment has the time started and time finished the meetup in the cabine. Your goal
is to find the minimum number of cabins that you need to meet all the meetings. You can only
book a cabine when is freed.

e.g.
segments = [[2,5], [1,3], [2,4]]
output = 3
"""


def find_minimum_cabines(segments: list[list[int]]) -> int:
    """In this approach we will use the dots methods, which in this case we will
    handle just a counter for how many people are currently in a meeting. For each meeting
    start we will add +1, and then if its ended, we will rest the -1. But we should do it
    chronologically, so first of all we should sort all the segments by time."""

    # Stage 1: Add to each time the state, if its free or busy
    times: list[list[int]] = []

    for segment in segments:
        times.append([segment[0], 1]) # We show that the person took started a meeting
        times.append([segment[1], -1]) # We show that the person ended the meeting

    # Stage 2: Sort the time chronologically
    times.sort()

    # Stage 3: Now we should iterate over the times and find the maximum concurrent meetings
    current: int = 0
    max_concurrent: int = 0

    for time in times:
        current += time[1]
        max_concurrent = max(max_concurrent, current)

    return max_concurrent


if __name__ == "__main__":
```

```
    segments: list[list[int]] = [[2, 5], [1, 3], [2, 4]]
    print(f"Segments: {segments}")
    response: int = find_minimum_cabines(segments)

    print(f"Minimum cabines needed: {response}")
```

3.2 Segments Method

Complexity

Time: O(n log n) **Memory:** O(n)

Green Flags -- When to Use Segments Method

- If we need merge, find intersections in a list of segments or time tables

Approach

When the question is about the intervals themselves – merging them, or finding the gaps between them – sort by start point so any two intervals that could overlap become neighbours in the sorted order, then make one left-to-right pass, extending or closing off the current merged segment as needed.

Pattern Template

```
"""
Pattern: segments method (merge overlapping intervals)

Time: O(n log n) -- dominated by the sort
Memory: O(n) -- for the sorted copy and the result list
"""


def merge(segments: list[list[int]]) -> list[list[int]]:
    # ================================================================================
    # Stage 1: Sort by start so any two segments that could overlap become
    # neighbours in the list
    # ================================================================================
    segments = sorted(segments)

    # ================================================================================
    # Stage 2: Seed the result with the first segment
    # ================================================================================
    result: list[list[int]] = [segments[0]]

    # ================================================================================
    # Stage 3: For every later segment, either extend the last merged
    # segment (it overlaps: its start is not past the last entry's end)
    # or start a brand new merged segment
    # ================================================================================
    for start, end in segments[1:]:
        ...

    return result
```

Worked Example

```python
"""
Given a list of segments, return a new list with all the interesected
segments unified into a single segment.

e.g.
segments = [[2,4], [1,3],[5,6], [6,7], [8,8]]
output: [[1,4],[5,7], [8,8]]

First of all we need to sort the segment by their starting point
"""


def is_intersection(s1: list[int], s2: list[int]) -> bool:
    return max(s1[0], s2[0]) <= min(s1[-1], s2[-1])


def merge_segments(s1: list[int], s2: list[int]) -> list[int]:
    return [min(s1[0], s2[0]), max(s1[-1], s2[-1])]


def unify_intersections(segments: list[list[int]]) -> list[list[int]]:
    if len(segments) < 2:
        return segments

    # Stage 1: Sorting the segments by their starting point
    segments.sort(key=lambda x: x[0])

    # Stage2: Initializing the first segment
    result: list[list[int]] = [segments[0]]

    for i in range(1, len(segments)):
        # Stage 3: Checking for intersections over the rest of the segments
        if is_intersection(result[-1], segments[i]):
            result[-1] = merge_segments(result[-1], segments[i])
        else:
            result.append(segments[i])

    return result


if __name__ == "__main__":
    seg1 = [[2, 4], [1, 3], [5, 6], [6, 7], [8, 8]]
    print(f"Segments: {seg1}\nUnified: {unify_intersections(seg1)}")
```

3.3 Two Pointers on Segments

Complexity

Time: O(n + m) **Memory:** O(max(n, m))

Green Flags -- When to Use Two Pointers on Segments

- If you see multiple segments with intervals

Approach

Two already-sorted lists of intervals can be intersected in a single linear pass: keep one pointer per list, record the overlap (if any) between the two intervals currently pointed to, and always advance whichever interval ends first, since it cannot possibly overlap anything further along in the other list.

Pattern Template

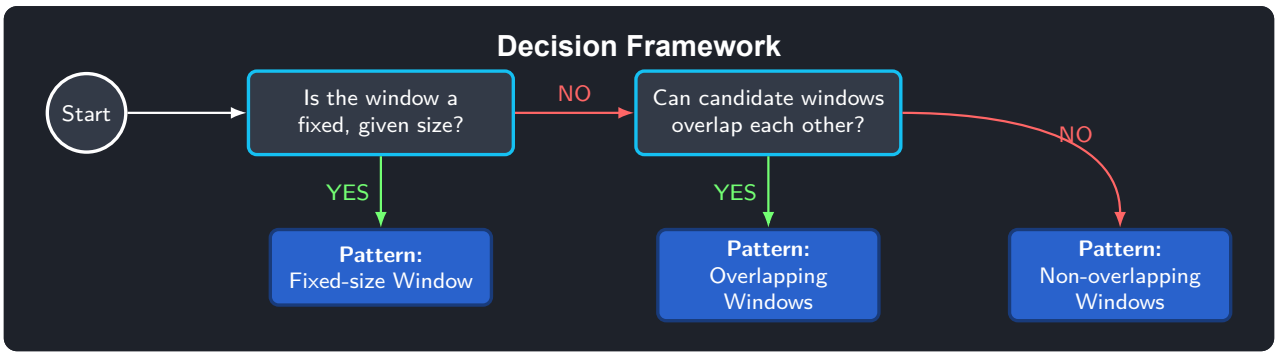
```
"""
Pattern: two pointers over two segment lists

Time: O(n + m)
Memory: O(max(n, m)) -- for the result list
"""


def multi_segment_intersect(seg1: list[list[int]], seg2: list[list[int]]) -> list[list[int]]:
    # ============================================================================
    # Stage 1: One pointer per already-sorted list, both starting at index 0
    # ============================================================================
    result: list[list[int]] = []
    p1, p2 = 0, 0

    # ============================================================================
    # Stage 2: Advance whichever segment ends first -- it cannot intersect
    # anything further to the right, so it is safe to discard
    # ============================================================================
    while p1 < len(seg1) and p2 < len(seg2):
        # ========================================================================
        # Stage 3: Record the overlap (if any) between seg1[p1] and seg2[p2],
        # then move the pointer whose segment ends first
        # ========================================================================
        ...

    return result
```

Worked Example

```
"""
Given two sorted lists of segments, we should find all the intersection between them,
between the two lists of given segments.

e.g.
seg1 = [[1,2],[3,4], [6,7]]
seg2 = [[1,5], [7,8]]

result: [[1,2],[3,4],[7,7]]

This approach should be using two pointers, one for each list of segments.
Then the approach is like in the pointer problems, but checking for intersections
"""


def intersection(seg1: list[int], seg2: list[int]) -> list[int]:
    start: int = max(seg1[0], seg2[0])
    end: int = min(seg1[-1], seg2[-1])

    if start <= end:
        return [start, end]
    else:
        return []


def multi_segment_intersections(
    seg1: list[list[int]], seg2: list[list[int]]
) -> list[list[int]]:
    if len(seg1) < 1 or len(seg2) < 1:
        return []

    # Stage 1: Initialize the pointers for the two lists and then check forth intersections
    p1: int = 0
    p2: int = 0
    result: list[list[int]] = []
```

```
    while p1 < len(seg1) and p2 < len(seg2):
        if intersect := intersection(seg1[p1], seg2[p2]):
            result.append(intersect)

        if seg1[p1][1] < seg2[p2][1]:
            p1 += 1
        else:
            p2 += 1

    return result


if __name__ == "__main__":
    seg1: list[list[int]] = [[1, 2], [3, 4], [6, 7]]
    seg2: list[list[int]] = [[1, 5], [7, 8]]

    print(
        f"Segments 1: {seg1}\nSegments 2: {seg2}\nIntersections: {multi_segment_intersections(seg1, seg2)}"
    )
```

Chapter 4
Sliding Window



4.1 Fixed-size Window

Complexity

Time: O(n) **Memory:** O(1)

Green Flags -- When to Use Fixed-size Window

- If you are provided with a k element which is a window size
- You need to work with followed numbers one on another

Approach

The window size k is fixed ahead of time, so maintain a running aggregate (sum, count, product, ...) over exactly the last k elements: add the incoming element, remove the element leaving the window k steps behind, and read off the aggregate at every position instead of recomputing it from scratch.

Pattern Template

```python
# Patterns for window of fixed length
# Time: O(n)
# Memory: O(1)


def elements_n_max_sum(nums: list[int], k: int) -> int:
    # ================================================================
    # 1) First filling of the window
    # ================================================================
    window_sum: int = 0
```

```
    for i in range(k):
        window_sum += nums[i]

    max_sum: int = window_sum

    # ================================================================================
    # 2) Loop for windown search
    # ================================================================================
    for r in range(k, len(nums)):
        # The formula of modifying the k elements between the window transitions,
        # which allows us to pass to the next window fastly
        # <This is the key to the solution>
        ...

    return max_sum
```

Worked Example

```
"""
Given a list of number and a number k, you should return the k numbers in the
list which sums the most.
"""

from os import path

DEFAULT_LIST: list[int] = [3, 2, 0, 9, 1, 2, 8, 5, 2] # Solution: 25
DEFAULT_NUMBER: int = 5


def read_input() -> tuple[list[int], int]:
    try:
        with open(path.join(path.dirname(__file__), "input.txt")) as f:
            inputs = list(map(int, f.readline().strip("\n").split(",") or DEFAULT_LIST))
            number = int(f.readline().strip("\n") or DEFAULT_NUMBER)
            return inputs, number

    except FileNotFoundError:
        return DEFAULT_LIST, DEFAULT_NUMBER


def max_sum_k_number(nums: list[int], k: int) -> int:
    """For this approach we will be using the sliding window approach"""
    window_sum: int = 0
    for i in range(k):
        window_sum += nums[i]

    max_sum = window_sum

    for r in range(k, len(nums)):
        window_sum = window_sum - nums[r - k] + nums[r]
        max_sum = max(
            max_sum,
            window_sum,
        )

    return max_sum


def max_sum_k_number_unoptimized(nums: list[int], k: int) -> int:
    """More typical approach, but IDK if it will be allowed in code platforms"""
    max_sum: int = 0

    for i in range(len(nums) - k + 1):
        max_sum = max(max_sum, sum(nums[i : i + k]))

    return max_sum

```

```python

if __name__ == "__main__":
    l, k = read_input()

    print(f"List: {l} | Number: {k}")
    max_sum = max_sum_k_number(l, k)
    print(f"Solution: {max_sum}")

    print(f"New list: {max_sum_k_number([3,2,0,9,1,2,8,5,-5,7,2], k)}")
```

4.2 Non-overlapping Windows

Complexity

Time: O(n) **Memory:** O(n)

Green Flags -- When to Use Non-overlapping Windows

- We need to work with following groups of elements
- The elements are not overlapping, one elements from one group is not part of another group

Approach

Once a window is closed off as a valid answer, none of its elements can be reused by a later window, so grow the current window until it becomes invalid (or the input ends), commit it, and restart the next window from the very next element – there is never a need to shrink from the left.

Pattern Template

```python
# Time: O(n)
# Memory: O(n)


def not_overlaping_windows(nums: list[int]) -> list[str]:
    # ================================================================================
    # 1) Left & Right initializations
    # ================================================================================
    left: int = 0
    right: int = 0
    groups: list[str] = []

    # ================================================================================
    # 2) Outer loop while checking all the conditions
    # ================================================================================
    while left < len(nums):
        # ============================================================================
        # 3) Expanding the right boundary while it meets all the conditions
        # Basically we need to handle here the condition of expanding the inner window
        # The problem is in this point, to know how to expand the window
        # ============================================================================
        while right + 1 < len(nums) and ...:
            right += 1

            # ========================================================================
            # 4) Working on the current inner window
            # ========================================================================
            ...

            # ========================================================================
```

```
            # 5) Changing to the next window when all the conditions are met.
            # Basically we do one more step from the last window pointer to start the next window
            # ============================================================================
            left = right + 1
            right = right + 1

        return groups
```

Worked Example

```
"""
Given a sorted list of integers. You should group it by grouping the
following numbers in the same group, if the next number is greater than
the previous in one, then they are in the same group. In each group you should add with the "->"
the followed numbers from the first and last of each group.

Example:
[1,2,3,5,8,9,14]
Solution: ["1->3", "5], "8->9", "14"]
"""


def format_group(*kwargs) -> str:
    if len(kwargs) > 1:
        return f"{kwargs[0]}->{kwargs[-1]}"
    return str(kwargs[0])


def group_numbers(nums: list[int]) -> list[str]:
    """The approach that we will be using is a not overlaping windows.

    Is a similar approach as in the slow and fast pointers, so basically we start with
    two pointers and we move them one step at a time, if the next number is greater
    than the previous in more than one, then this is one group.
    """
    groups: list[str] = []

    slow: int = 0
    fast: int = 0

    while slow < len(nums):
        # Now we need to expand the right boundary while it means the condition
        while fast + 1 < len(nums) and nums[fast] + 1 == nums[fast + 1]:
            fast += 1

        # If the boundary stopped expanding, what we need to do is to add the group now
        if fast != slow:
            groups.append(format_group(nums[slow], nums[fast]))
        else:
            groups.append(format_group(nums[slow]))

        slow = fast + 1
        fast = fast + 1

    return groups


if __name__ == "__main__":
    EXAMPLE_LIST: list[int] = [1, 2, 3, 5, 8, 9, 13, 14, 16]
    print(f"Example list: {EXAMPLE_LIST}")
    res = group_numbers(EXAMPLE_LIST)
    print(f"Solution: {res}")
```

4.3 Overlapping Windows

Complexity

Time: O(n) **Memory:** O(1)

Green Flags -- When to Use Overlapping Windows

- We need to work with following groups of elements
- One elements is part of one group (the groups are not overlaping)

Approach

Grow the window from the right for as long as it stays valid; the moment it becomes invalid, shrink it from the left – just enough to make it valid again, never resetting it – then keep growing. Because both pointers only ever move forward, the whole scan stays linear even though many overlapping windows are implicitly considered.

Pattern Template

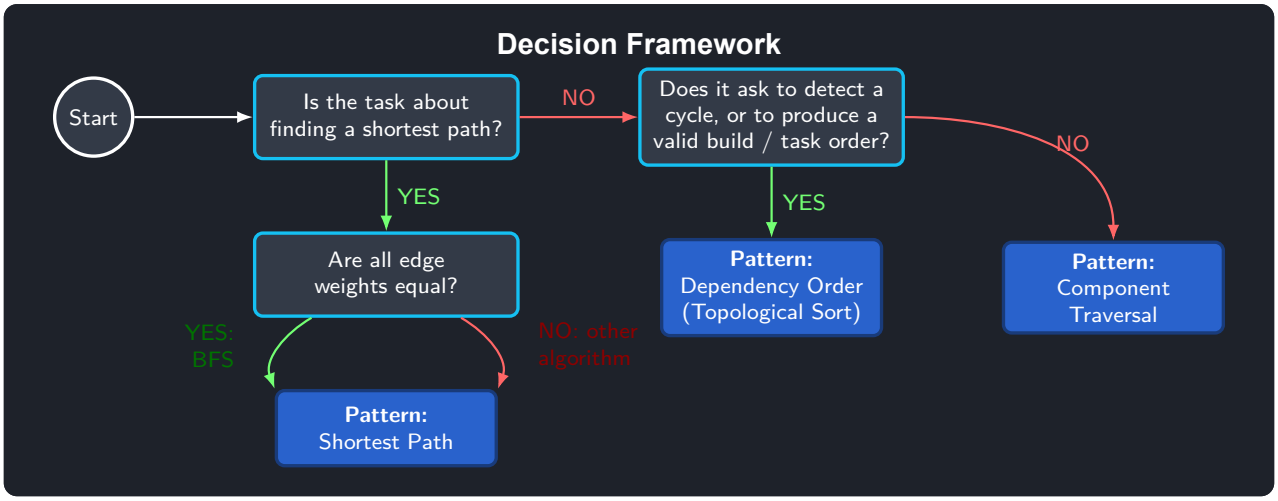
```python
"""
The idea is to basically use two pointers too, then we add a counter
store the changes we are making on the window. So we start moving
right pointer, if there is a zero or the respective number, we only
move the pointer if the counter is less than the number. So we
increase the pointer while we are not at the end of the counter
is greater than the number of zeros in the window.
"""

# Time: O(n)
# Memory: O(1)


def longest_ones_with_flips(nums: list[int], k: int) -> int:
    # ============================================================================
    # 1) Left & Right initializations
    # ============================================================================

    left: int = 0
    right: int = -1 # In order to not process the first element independently
    result: int = 0

    # ============================================================================
    # 2) Initializing the window state. THIS IS THE MAIN PROBLEM
    # You need to understand what is the real state of the window
    # ============================================================================
    ...

    # ============================================================================
    # 3) Outer loop
    # ============================================================================
    while left < len(nums):
        # ========================================================================
        # 4) Widering window loop
        # ========================================================================
        while right + 1 < len(nums) and ...: # THINK ABOUT THE EXPANSION STATE
            # updating the window state
            right += 1
            ... # WINDOW STATE UPDATE TOO

        # ========================================================================
        # 5) Renewind the result
```

```
            # ================================================================================
            result = max(result, right - left + 1)

            # ================================================================================
            # 6) Window decrementation
            # ================================================================================
            ... # STATUS UPDATE
            left += 1
        return result
```

Worked Example

```
"""
Given a list of numbers of 0s and 1s, and a value k, you are allowed
to change the k times zeros to ones. You should find the maximum number of the
following ones in the list.

e.g.
nums = [1,0,1,0,1,0,1,1]
k = 2

res = 6
"""


def max_ones_with_changes(nums: list[int], k: int) -> int:
    left: int = 0
    right: int = -1
    result: int = 0

    zeroes_counter: int = 0

    while left < len(nums):
        while right + 1 < len(nums) and (nums[right + 1] == 1 or zeroes_counter < k):
            if nums[right + 1] == 0:
                zeroes_counter += 1
            right += 1

        # Now we check the maximum result
        result = max(result, right - left + 1)

        if nums[left] == 0:
            zeroes_counter -= 1
        left += 1

    return result


if __name__ == "__main__":
    input_list: list[int] = [0, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1]

    print(f"Input: {input_list}")
    result = max_ones_with_changes(nums=input_list, k=2)
    print(f"Solution: {result}")
```

Chapter 5
Graphs



Graph Traversal Tips

Both graph traversals share the same skeleton: a `visited` set plus one open container of "nodes still to explore" – the only difference is which end of that container is removed from.

- **BFS** removes from the *front* (a queue) and explores level by level, so the first time it reaches a node is guaranteed to be via a shortest path (in edge count, or in total weight when every edge costs the same).
- **DFS** removes from the *top* (a stack, often the call stack via recursion) and commits to one path as deep as possible before backtracking, which makes it a natural fit for existence/path-construction questions rather than distance questions.

If the graph is a grid or maze, encode moves as a small list of $(\Delta r, \Delta c)$ direction deltas and guard every step with an `in_bounds` check – this keeps the exploration loop identical no matter how many directions are legal.
Both traversals share the same asymptotic bounds, $O(V + E)$ time and memory, since every vertex and edge is visited at most once; they mainly differ in constant-factor memory behaviour and in which questions they answer well:

Property	BFS	DFS
Big-O time & memory	same	same
Typical memory use	lower (frontier)	higher (deep stack)
Best suited for	shortest distance	path existence / structure

5.1 Connected Components Traversal

Green Flags -- When to Use Connected Components Traversal

- We need to find the number of connected components in the graph.
- We need to find the maximum/minmum components with some property (e.g. Components with the largest sum of values).

Approach

Repeatedly launch a traversal (BFS or DFS – the choice barely matters here) from any unvisited node, marking every node it reaches as visited; each traversal launched corresponds to exactly one connected component. Counting how many traversals were started answers "how many components" questions directly, and comparing what each traversal collects answers "which component has property X" questions.

Pattern Template

```python
"""
Pattern connected components traversal

Time: O()
Memory: O()

"""


def count_connected_components(edges: list[list[int]]) -> int:
    # ================================================================================
    # Stage 1: Form the graph if needed
    # ================================================================================
    graph = ...

    # ================================================================================
    # Stage 2: Initialize the shared variable for the multiple loops
    # ================================================================================
    visited: set = set()
    components: int = 0

    # ================================================================================
    # Stage 3: Loop starting login and processing of the results
    # ================================================================================
    ...

    return components
```

Worked Example

```python
"""
Given a not oriented graph as a form of edges, we need to find the number of connected components it has.

e.g.
graph = [[11,4],[4,12],[4,10],[12,10], [21,23],[1,2],[1,3]]

result = 3
"""

from collections import deque, defaultdict
from pprint import pprint
```

```python


def build_graph(edges: list[list[int]]) -> tuple[dict[int, list[int]], set[int]]:
    graph: defaultdict[int, list[int]] = defaultdict(list)
    vertices = set()
    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)
        vertices.add(u)
        vertices.add(v)

    return graph, vertices


def bfs(start: int, graph: dict[int, list[int]], visited: set[int]) -> None:
    """
    We wil be doying the BFS here, and it will set the visited nodes, so basically
    every node visited will be added to the visited set, therefore if some nodes are not visited,
    it means that they are part of another component.
    """
    queue: deque[int] = deque([start])
    while queue:
        node: int = queue.popleft()

        for neighbour in graph.get(node, []):
            if neighbour not in visited:
                queue.append(neighbour)
                visited.add(neighbour)


def count_connected_components(edges: list[list[int]]) -> int:
    # Stage 1: BUild the graph
    graph, vertices = build_graph(edges)

    # Stage 2: Initialize the shared variables for the multiple loops
    visited: set[int] = set()
    components: int = 0

    for node in vertices:
        # If is not visited, it means that it is a new different component
        if node not in visited:
            bfs(node, graph, visited)
            components += 1

    pprint(graph)

    return components


if __name__ == "__main__":
    graph = [[11, 4], [4, 12], [4, 10], [12, 10], [21, 23], [1, 2], [1, 3]]
    print(f"Graph: {graph}")
    print(f"Number of connected components: {count_connected_components(graph)}")
```

5.2 Dependency Order (Topological Sort)

Complexity

Time: O(V + E) **Memory:** O(V + E)

Green Flags -- When to Use Dependency Order (Topological Sort)

This sub-pattern has no dedicated notes yet in the source repository; refer to the explanation and pattern code below.

Approach

Model prerequisites as directed edges and count, for every node, how many prerequisites it is still waiting on (its in-degree). Nodes that start at in-degree zero can run immediately; process them, decrement the in-degree of everything they unlock, and enqueue any node that reaches zero. If the queue empties before every node has been processed, a cycle exists and no valid order is possible.

Pattern Template

```python
"""
Pattern: dependency order (topological sort)

Time: O(V + E)
Memory: O(V + E) -- adjacency list, in-degree array and the queue

Only valid on a Directed Acyclic Graph (DAG): if a cycle exists, no
valid order exists, and this returns fewer than num_tasks entries.
"""

from collections import deque, defaultdict


def topological_order(num_tasks: int, prerequisites: list[list[int]]) -> list[int]:
    # ================================================================================
    # Stage 1: Build the graph and count how many prerequisites each task
    # is still waiting on (its in-degree)
    # ================================================================================
    graph: dict[int, list[int]] = defaultdict(list)
    in_degree = [0] * num_tasks
    for prerequisite, task in prerequisites:
        graph[prerequisite].append(task)
        in_degree[task] += 1

    # ================================================================================
    # Stage 2: Every task with in-degree 0 can run right now -- seed the
    # queue with all of them (this is plain BFS / Kahn's algorithm)
    # ================================================================================
    queue = deque(task for task in range(num_tasks) if in_degree[task] == 0)
    order: list[int] = []

    # ================================================================================
    # Stage 3: "Remove" each processed task by decrementing its
    # dependents' in-degree; a dependent becomes runnable once it reaches 0
    # ================================================================================
    while queue:
        task = queue.popleft()
        order.append(task)
        for dependent in graph[task]:
            in_degree[dependent] -= 1
            if in_degree[dependent] == 0:
                queue.append(dependent)

    # ================================================================================
    # Stage 4: If not every task was reached, a cycle blocked the rest
    # ================================================================================
    return order if len(order) == num_tasks else []
```

Worked Example

```python
"""
Given a graph representing some tasks, where each task has some dependencies before it.
You should find the order in which the tasks should be executed to make
sure that all the dependencies are met. If there are many responses, we can return
the one we want. If there is a cycle, we should return an empty list.
```

```
e.g.

graph= [[2,1],[9,1],[9,8],[1,7], [1,8], [7],[8],[11,4],[4,8]]

response = [2,9,11,1,4,7,8 ]

That problem is a topology problem, so we can solve it by using the Kahn algorithm.
Like a perceptron, we count first the number of nodes entering each node, aka in-degree.


"""


def dependency_order(graph: list[list[int]]) -> list[int]: ...


if __name__ == "__main__":
    graph: list[list[int]] = [
        [2, 1],
        [9, 1],
        [9, 8],
        [1, 7],
        [1, 8],
        [7],
        [8],
        [11, 4],
        [4, 8],
    ]
    print(f"Graph: {graph}")
    print(f"Order: {dependency_order(graph)}")
```

5.3 Shortest Path

Complexity

Time: O(n*m) **Memory:** O(n*m)

Green Flags -- When to Use Shortest Path

- If we need to find the minimum number of edges/path length/ generate path....

Approach

When every edge has the same cost, BFS already produces shortest paths for free, because it explores the graph strictly in order of distance from the source – no separate shortest-path machinery is needed, just the standard BFS traversal with a distance counter carried alongside each queued node.

Pattern Template

```
"""
Time: O(n*m)
Memory: O(n*m)
"""

from collections import deque


def in_bounds(x: int, y: int, graph: list[list[int]]) -> bool:
```

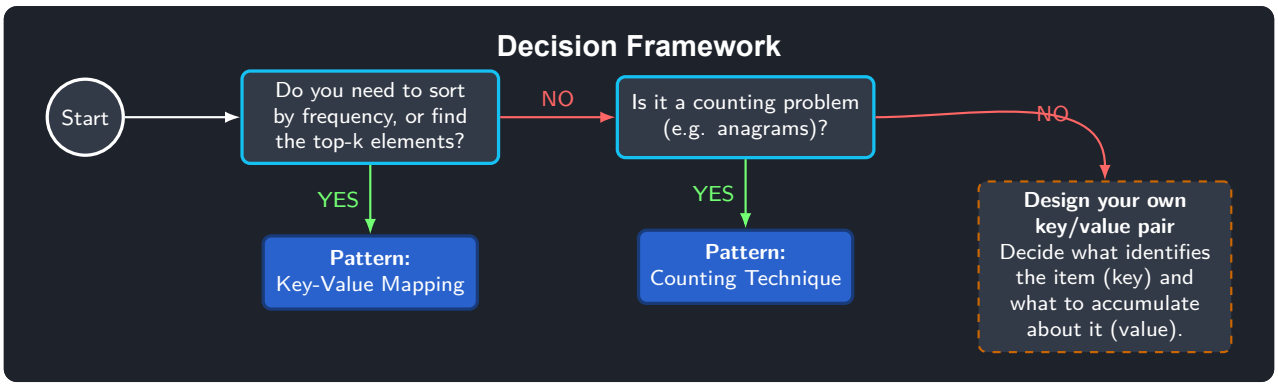
```
    return 0 <= x < len(graph) and 0 <= y < len(graph[0])


def find_start(graph: list[list[int]]) -> list[int]:
    """
    If we need to start randomly in the graph
    """
    for row in range(len(graph)):
        for col in range(len(graph[0])):
            if graph[row][col] == 2:
                return [row, col]
    return [-1, -1]


def shortest_path(graph: list[list[int]]) -> int:
    # ========================================================================
    # Stage 1: Where is BFS starting=?
    # Main problem is to figure out that we need to use the bfs
    # ========================================================================
    start = ...

    queue: deque = deque(
        [(start[0], start[1], 0)]
    ) # Counters are added too if needed :)
    visited: set[tuple[int, int]] = {
        (start[0], start[1])
    } # Remember your last girlfriend and don't walk nearby :)

    directions: list[tuple[int, int]] = [
        (0, 1),
        (0, -1),
        (1, 0),
        (-1, 0),
    ] # This is just for this types of graphs

    while queue:
        row, col, dist = queue.popleft()

        # ========================================================================
        # Stage 2: Processing the current path, node and result basically
        # ========================================================================
        ...

        # ========================================================================
        # Stage 3: Loop over your neightbours
        # ========================================================================
        for dr, dc in directions:
            new_row = row + dr
            new_col = col + dc

            # ========================================================================
            # Stage 4: Decide which neighbours to add to the queue
            # ========================================================================
            if ...:
                queue.append((new_row, new_col, dist + 1))
                visited.add((new_row, new_col))

    return -1 # If there are no path
```

Worked Example

```
"""
We are provided with an oriented graph as a list of edges, where each edge is a pair [u, v]
and represents the edge from vertex u to vertex v. We are also provided with a starting point start and
last point finish. We need to find the shortest path from start to finish. If the path does not exist,
then return -1.

```

```
e.g.
edges = [[0,1], [0,2], [1,3], [2,3],[3,4]]
start = 0
finish = 4

response = 2
"""

from collections import deque
from pprint import pprint
from patterns.graphs.lib import graph_from_edges


def find_shortest_path(graph: dict[int, list[int]], start: int, finish: int) -> int:
    # We will be using BFS here too, also with the counter for the distance
    queue: deque[tuple[int, int]] = deque(
        [(start, 0)]
    ) # Start represents the current node where we are starting
    visited: set[int] = {
        start
    } # Visited nodes, so basically we are doing the typical BFS

    # Directions are not needed since we are using a normal graph and not some special representation

    while queue:
        # get the first element from the list and process it
        node, distance = queue.popleft()

        # Check if we are at the end or not
        if node == finish:
            return distance

        # If we are not at the end, we should check all the neighbours to the current node possible
        for neighbour in graph.get(node, []):
            # Check if they are visited or not
            if neighbour not in visited:
                queue.append((neighbour, distance + 1))

    return -1


if __name__ == "__main__":
    edges: list[list[int]] = [[0, 1], [0, 2], [1, 3], [2, 3], [3, 4]]
    start: int = 0
    finish: int = 4

    graph: dict[int, list[int]] = graph_from_edges(edges, is_directed=True)
    pprint(graph)
    print(
        f"Shortest path from {start} to {finish}: {find_shortest_path(graph, start, finish)}"
    )
```

Chapter 6
Hash Tables



6.1 Counting Technique

Complexity

Time: O(2n) -> O(n) **Memory:** O(k) where k is the number of unique chars

Green Flags -- When to Use Counting Technique

- Anagram problems
- Working with an element frequency

Approach

Reduce the problem to comparing frequency profiles instead of comparing raw sequences: count how many times each element occurs, and answer the question purely from those counts (equal counts mean equal multisets, an odd count for at most one element means a palindrome rearrangement is possible, and so on) rather than by manipulating the original data directly.

Pattern Template

```
# Time: O(2n) -> O(n)
# Memory: O(k) where k is the number of unique chars


def is_palindrome_permutation(chars: list) -> bool:
    # ================================================================
    # 1) Counter for every char in the string
```

```
    # ============================================================================
    count: dict[str, int] = {}

    for char in chars:
        # Counting every char frequency
        ...

    # ============================================================================
    # 2) Processing if the needed condition is possible
    # ============================================================================
    ...
```

Worked Example

```
"""
Given a list of chars chars. You should return tru if is possible to
re-arrange its chars the way that we obtain a palindrome.
"""


def can_make_palindrome(chars: list[str]) -> bool:
    """This approach will strictly follow the counting technique of hash tables.
    We should know things like that the palindrome is formed only when has all the letters
    even number of times or when all are even and only one is odd.
    """
    char_counter: dict[str, int] = {}

    # Go trough all the chars and count them one by one O(n)
    for c in chars:
        char_counter[c] = char_counter.get(c, 0) + 1

    # Now we check if the palindrome is possible by looking how many time we have odd and even.
    odd_counter: int = 0
    for v in char_counter.values():
        if v % 2 == 1:
            odd_counter += 1

    return odd_counter <= 1


if __name__ == "__main__":
    true_chars: list[str] = ["a", "b", "a", "a", "b", "c", "c"]
    false_chars: list[str] = ["a", "b", "c"]
    print(f"Chars: {true_chars} -> Solution: {can_make_palindrome(true_chars)}")
    print(f"Chars: {false_chars} -> Solution: {can_make_palindrome(false_chars)}")
```

6.2 Key-Value Mapping

Complexity

Time: O(n) **Memory:** O(n)

Green Flags -- When to Use Key-Value Mapping

- Problem for filtering by frequency
- We need to search for the top-k by some specific criteria

Approach

Pick, for the specific problem, what should be the key and what should be the value – often the key is the item's identity or category and the value is an aggregate about it (its frequency, its last-seen index, a list of items sharing that key). Once that choice is made, most of these problems reduce to one linear pass that builds the map, followed by a second pass that reads answers back out of it.

Pattern Template

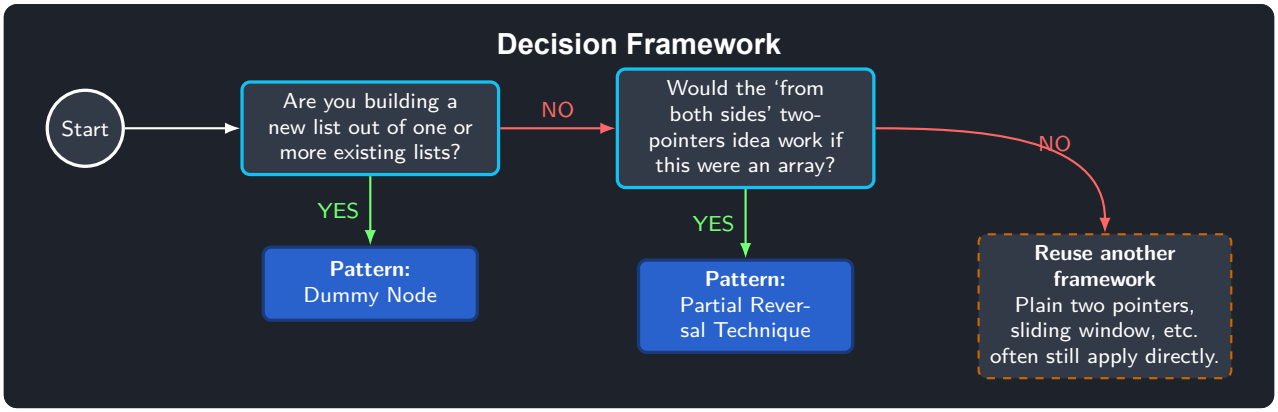
```python
# Time: O(n)
# Memory: O(n)


def frequency_sort(chars: str) -> str:
    # Stage 1: KV -> Basically doying the counting
    count: dict[str, int] = {} # Key:Value -> Char:Frequency
    for char in chars:
        ...

    # Stage 2: VK (Inversion of frequencies)
    # CORE PROBLEM: Understand that the problem needs the inversion, if you know that
    # then you can solve it easily
    frequency_list: list[list[str]] = [
        [] for _ in range(len(chars) + 1) # Index: frequency -> quantity: symbols
    ]

    for char, freq in count.items():
        ...

    # Stage 3: Grouping the result
    result: list[str] = []
    ...

    return "".join(result)
```

Worked Example

```python
"""
Given a list with characters. you should sort its symbols by their frequency,
from the most frequent to the least frequent.

e.g.
chars = ["B", "A", "B", "B", "C", "B", "C"]
Solution = BBBBCCA

"""


def sort_by_frequency(chars: list[str]) -> str:
    """This approach should only use th ehash tables key value pattern"""

    # First pass, counting the frequency of each char
    counter: dict[str, int] = {}
    for char in chars:
        counter[char] = counter.get(char, 0) + 1

    # Second pass, we create the frequency list
    frequency_list: list[list[str]] = [[] for _ in range(len(chars) + 1)]

    for char, freq in counter.items():
        frequency_list[freq].append(char)

    # Create the final responmse list with the sorted chars
    result: list[str] = []
```

```
    for i in range(len(frequency_list) - 1, 0, -1):
        for char in frequency_list[i]:
            result.append(char * i)

    return "".join(result)


if __name__ == "__main__":
    input_list: list[str] = ["B", "A", "B", "B", "C", "B", "C"]

    result: str = sort_by_frequency(input_list)
    print(f"Input: {input_list} | Solution: {result}")
```

Chapter 7
Linked Lists



Linked List Tips

Whenever an algorithm compares or reconstructs *two halves* of a singly linked list around its middle node, the key question is whether the two traversals will only *read* node values, or will also *rewrite* `next` pointers.

If both halves only read values (e.g. checking a palindrome), it is safe to let both pointers reach the shared middle node – following an existing link never creates a cycle. But the moment a traversal reassigns `next` (merging, interleaving, reordering), two pointers that can still reach the same physical node can end up creating one:

`curr.next = ptr_2` can silently become `node.next = node`.

The fix is to physically cut the list before rewiring:

```
second_half = middle.next
middle.next = None # detach -- this does not delete any node
second_half = reverse(second_half)
```

This frees no memory; it only removes the link between the two halves so each can be reversed, merged or interleaved independently, with no risk of an accidental cycle.

Situation	Cut needed?
Only reading both halves	usually not
Rewiring / merging / interleaving	yes, always cut first

7.1 Basic Operations

Complexity

Time: O(n) **Memory:** O(1)

Green Flags -- When to Use Basic Operations

This sub-pattern has no dedicated notes yet in the source repository; refer to the explanation and pattern code below.

Approach

Every more advanced linked-list pattern in this book leans on the same handful of primitives: computing the length, finding the middle node (typically with a fast/slow pointer pair), and reversing a sublist in place by walking it once while re-pointing each node's `next` backwards. Being fluent with these O(1)-memory building blocks is what makes the more advanced patterns feel like composition rather than new algorithms.

Pattern Template

```
"""
Pattern: basic linked-list operations

These are the small building blocks (size, middle, reverse) that every
other linked-list pattern in this book is assembled from.

Time: O(n)
Memory: O(1)
"""
# Node here is a plain singly linked node with .data and .next attributes.


def reverse(head: "Node") -> "Node":
    # ================================================================================
    # Stage 1: prev starts at None -- that becomes the new tail's `next`
    # ================================================================================
    prev = None
    curr = head

    # ================================================================================
    # Stage 2: At each node, save `next` before overwriting it, flip the
    # pointer to point backwards, then advance both prev and curr
    # ================================================================================
    while curr is not None:
        next_node = curr.next
        curr.next = prev
        prev = curr
        curr = next_node

    # ================================================================================
    # Stage 3: prev now points at the old tail -- the new head
    # ================================================================================
    return prev
```

Worked Example

```
from patterns.linked_lists.lib import Node

```

```
"""
We need 3 pointers:
- prev: to point to the previous node
- curr: to point to the current node
- tmp: to point to the next node


Time: O(n)
Memory: O(1)
"""


def reverse(head: Node) -> Node:
    prev: Node = None
    curr: Node = head

    while curr is not None:
        # Save the curr
        tmp: Node = curr

        # Move the curr to the next
        curr = curr.next

        # Break the pointer in order to point to reversed list
        tmp.next = prev

        # Point to the other side
        prev = tmp

    return prev


if __name__ == "__main__":
    linked_list: Node = Node(1, Node(2, Node(3, Node(4, Node(5, None)))))

    reversed_list: Node = reverse(linked_list)

    curr = reversed_list
    print(f"Reversed list:")
    l_str: str = ""
    while curr is not None:
        l_str += f"{curr.data} -> "
        curr = curr.next
    print(l_str)
```

7.2 Dummy Node

Complexity

Time: O(n + m) **Memory:** O(1)

Green Flags -- When to Use Dummy Node

- Node is needed to store the result
- If we want to process some deletion operation on the first node, etc.

Approach

Whenever the output is itself a newly assembled linked list, attach nodes to a throwaway dummy head instead of special-casing "what is the first node". The dummy node is discarded at the end by simply returning `dummy.next`, which keeps the main loop free of first-node edge cases.

Pattern Template

```python
"""
Pattern: dummy node (build a new linked list)

A dummy head removes the special case of "the first node doesn't
exist yet": we always attach to dummy.next and return that at the end.

Time: O(n + m)
Memory: O(1) -- the answer reuses the input nodes, nothing extra is copied
"""


def merge_lists(head_1: "Node", head_2: "Node") -> "Node":
    # ============================================================================
    # Stage 1: A throwaway dummy node so the first attached node never
    # needs special-casing
    # ============================================================================
    dummy = Node(0, None)
    curr = dummy

    # ============================================================================
    # Stage 2: Walk both lists, always attaching the next node the answer
    # needs -- typically combined with the two-pointers or sliding-window
    # approach to decide which one that is
    # ============================================================================
    while head_1 is not None and head_2 is not None:
        ...

    # ============================================================================
    # Stage 3: Return dummy.next, the real head of the built list
    # (the dummy node itself is discarded)
    # ============================================================================
    return dummy.next
```

Worked Example

```python
from patterns.linked_lists.lib import Node

"""
Given two linked lists, we need to merge them in a single ordered list and return the head.

e.g.
list1 = 1->2->3
list2 = 2->2->7->8

output = 1->2->2->2->3->7->8

The idea here is to create a dummy node, and them apply the pattern pointer for every one.
"""


def merge_lists(ptr_1: Node, ptr_2: Node) -> Node:
    # Stage 1: Initialization
    dummy_node: Node = Node(0, None) # Will be used as a result
    curr: Node = dummy_node # Goes through the list

    # Stage 2: Loop over the lists and merge them using the well know pattern pointer for everyone
    while ptr_1 is not None or ptr_2 is not None:
        if ptr_2 is None or (ptr_1 is not None and ptr_1.data <= ptr_2.data):
            curr.next = ptr_1
            ptr_1 = ptr_1.next
        else:
            curr.next = ptr_2
            ptr_2 = ptr_2.next
        curr = curr.next

```

```
    # Stage 3: Delete the dummy node by returnin the next pointer of himself
    return dummy_node.next


if __name__ == "__main__":
    list_1: Node = Node(1, Node(2, Node(3, None)))
    list_2: Node = Node(2, Node(2, Node(7, Node(8, None))))

    merged_list: Node = merge_lists(list_1, list_2)
    print(f"Merged list:")
    s_str: str = ""
    while merged_list is not None:
        s_str += f"{merged_list.data} -> "
        merged_list = merged_list.next
    print(s_str)

    list_3: Node = Node(2, Node(2, Node(7, Node(9, None))))
    list_4: Node = Node(1, Node(2, Node(7, Node(8, Node(9, None)))))
    merged_list_2: Node = merge_lists(list_3, list_4)
    print(f"\nMerged list 2:")
    s_str = ""
    while merged_list_2 is not None:
        s_str += f"{merged_list_2.data} -> "
        merged_list_2 = merged_list_2.next
    print(s_str)
```

7.3 Partial Reversal Technique

Complexity

Time: O(n) **Memory:** O(1)

Green Flags -- When to Use Partial Reversal Technique

- If the ”from both sides” patters seems to work here, in the linked list, then we definitely should try this approach
- Working with palindromes in a linked list

Approach

A singly linked list has no backward pointer, so the “two pointers from both ends” idea cannot be applied directly. Work around this by reversing the back half of the list in place first – turning it into its own independent, outside-in chain – and then walk both halves simultaneously from their new outer ends towards the middle, exactly like the plain two-pointers pattern.

Pattern Template

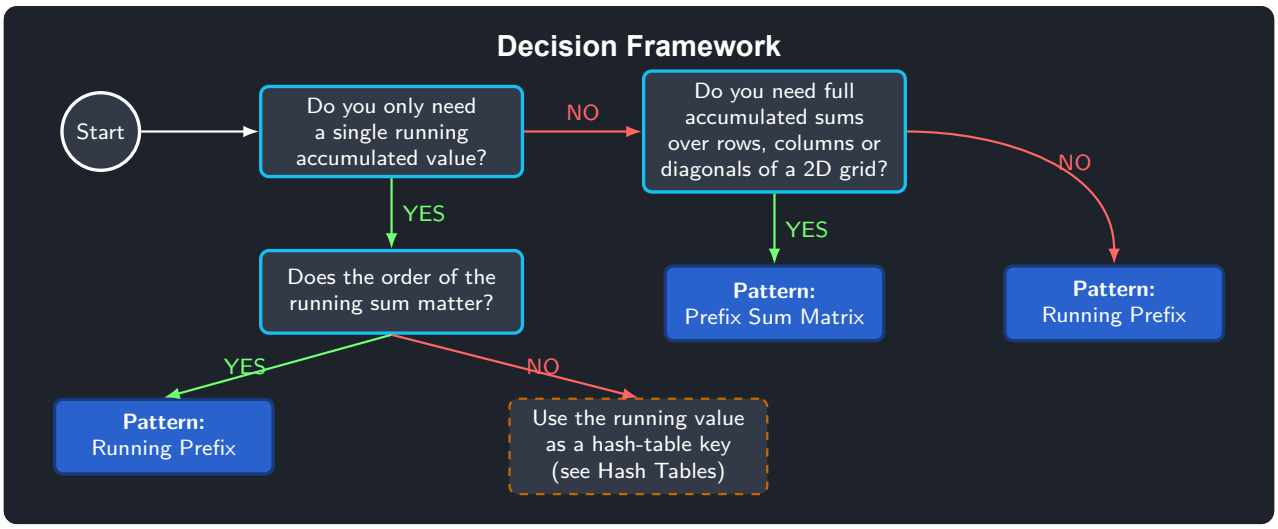
```
"""
Pattern: partial reversal technique

Time: O(n)
Memory: O(1) -- reversal happens in place, no extra list is allocated

A singly linked list has no "walk backwards" pointer, so the "from both
sides" two-pointers idea needs one trick to work here.
"""


def is_palindrome(head: "Node") -> bool:
```

```
    # ================================================================================
    # Stage 1: Find the middle, then reverse the second half in place.
    #
    # 1 -> 2 -> 3 <- 2 <- 1
    # |
    # v
    # None
    #
    # The middle node's `next` now points to None, giving two independent
    # chains that both run "outside-in".
    # ================================================================================
    middle = ...
    second_half_reversed = ...

    # ================================================================================
    # Stage 2: Walk both halves from the outside towards the middle,
    # comparing values -- the same two-pointers idea, just applied to two
    # separate chains instead of one list with two ends
    # ================================================================================
    ptr_1, ptr_2 = head, second_half_reversed
    while ptr_2 is not None:
        ...

    return True
```

Worked Example

```
"""
We have a linked list of int numbers, and we need to check if this linked list
is a palindrome.

e.g.
list = 1->2->3->2->1
output = True

list = 1->2->3->4->5
output = False
"""

from patterns.linked_lists.lib import Node
from patterns.linked_lists.basic_operations.list_reverse import reverse
from patterns.linked_lists.basic_operations.list_middle import middle


# The main problem here is to correctly choose the helper pattern functions (middle, reverse...)
def is_palindrome(head: Node) -> bool:
    # Stage 1
    first_half_end: Node = middle(head)
    second_half_begin = reverse(first_half_end)
    ptr_1, ptr_2 = head, second_half_begin

    while ptr_2:
        if ptr_1.data != ptr_2.data:
            return False

        ptr_1 = ptr_1.next
        ptr_2 = ptr_2.next

    return True


if __name__ == "__main__":
    linked_list: Node = Node(1, Node(2, Node(3, Node(2, Node(1, None)))))
    bad_linked_list: Node = Node(1, Node(2, Node(3, Node(4, Node(5, None)))))

    print(f"LInked list 1->2->3->2->1: {is_palindrome(linked_list)}")
    print(f"LInked list 1->2->3->4->5: {is_palindrome(bad_linked_list)}")
```

Chapter 8
Prefix Sums



8.1 Running Prefix

Complexity

Time: O(n) **Memory:** O(1)

Green Flags -- When to Use Running Prefix

- We don't need to store all prefixes, but we allowed to have just a running summs/products etc (accumulative)
- The problem needs a prefix and suffix from the list,but can be changed with variables
- Important: The running pivot can be a key from a hash table and be used in other patterns. e.g. Trees

Approach

Maintain a single running total while scanning the list once, and derive whatever is needed about "the rest of the array" by subtracting the running total from the overall total computed up front – this avoids ever looking ahead or recomputing a sum from scratch, keeping both time and extra memory linear-to-constant.

Pattern Template

```python
"""
Pattern: running prefix sum

Time: O(n)
Memory: O(1) -- one running total, no prefix array is stored
"""


def running_prefix(nums: list[int]) -> int:
    # ============================================================================
    # Stage 1: The total sum gives the "right side" sum for free:
    # right_sum = total_sum - left_sum - current, so nothing needs to be
    # looked up ahead of the current position
    # ============================================================================
    total_sum = sum(nums)
    left_sum = 0

    # ============================================================================
    # Stage 2: Slide across the list, updating left_sum as each element is
    # consumed, and compare it against the implied right_sum
    # ============================================================================
    for num in nums:
        # ========================================================================
        # Stage 3: Update left_sum / right_sum here and check the condition
        # asked for (equality, max difference, pivot index, ...)
        # ========================================================================
        ...

    return -1
```

Worked Example

```python
"""
Given a list of numbers, we need to find an elemnt where all the sums of numbers
before it are equal to the sum of the elements after it.

e.g.
nums = [7,-1, 4, 10, 5, 5]

result = 3.

Since we have the sums before it = 10 and after it also equals 10.
"""


def find_equal_sum(nums: list[int]) -> int:
    # Stage 1: Initialization
    total_sum: int = sum(nums)
    left_sum: int = 0

    # Stage 2: While loop while seaching for the response
    for i, num in enumerate(nums):
        if left_sum == total_sum - num - left_sum:
            return i

        left_sum += num

    return -1


if __name__ == "__main__":
    nums: list[int] = [7, -1, 4, 10, 5, 5]

    response: int = find_equal_sum(nums)
    print(f"Nums: {nums} -> Equal sum: {response}")
```

8.2 Prefix Sum Matrix

Complexity

Time: O(n*m) **Memory:** O(n+m)

Green Flags -- When to Use Prefix Sum Matrix

- Problems for chess and other games with max of sums
- You need to calculate sub numbers inside a multidimentional array/ tensor (sums, products, xor ops, etc)

Tips

When a 2D prefix-sum problem needs to group cells by *diagonal* rather than by row or column, the trick is to find an expression that stays constant for every cell on the same diagonal.
For a cell (i, j), moving one step along the descending diagonal ($\searrow$) takes $(i, j) \to (i + 1, j + 1)$, so $i - j$ is invariant along it; since $i - j$ can be negative, shift it into a valid array index using the column count:

$$\text{index}_{\searrow} = i - j + (\text{cols} - 1).$$

Moving one step along the ascending diagonal ($\nearrow$) takes $(i, j) \to (i+1, j-1)$, so $i+j$ is invariant along it and is already non-negative:

$$\text{index}_{\nearrow} = i + j.$$

Once every cell is mapped to its diagonal index, the running sum for that diagonal can be accumulated in a single 1D array positioned at that index – exactly the same running-prefix idea used for rows and columns, just applied along a different axis.

Approach

Extend the 1D running-sum idea to two dimensions by building a prefix matrix where each cell stores the accumulated sum of the rectangle above and to the left of it; any sub-rectangle's sum then follows from four cell lookups and an inclusion-exclusion subtraction, without ever re-summing the rectangle's interior.

Pattern Template

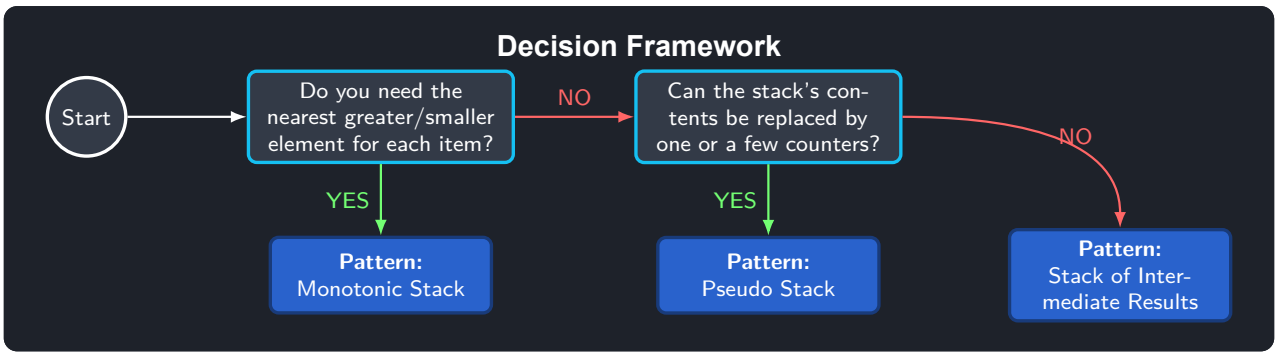
```python
"""
Sum matrix
Time: O(n*m)
Memory: O(n+m)
"""


def max_rook_sum(board: list[list[int]]) -> int:
    # ================================================================================
    # Stage 1: Initialization
    # ================================================================================
    n: int = len(board)
    m: int = len(board[0])

    # The base problem is to know how many sum matrices we need and which ones
    rows_sum: list[int] = [0] * n
    cols_sum: list[int] = [0] * m

```

```
    # ================================================================================
    # Stage 2: Sum rows and cols, create a prefix list basically
    # ================================================================================
    for i in range(n):
        for j in range(m):
            ...

    # ================================================================================
    # Stage 3: Search for the response: Use the pre-computed sums to find the
    # solution in an optimal way.
    # ================================================================================
    max_sum: int = 0
    for i in range(n):
        for j in range(m):
            # Sum calculations for the position [i,j]
            ...

    return max_sum
```

Worked Example

```
"""
Given a matrix of int nuymbers, find the max sum.

e.g.
matrix:
1,5,1,-1
3,1,2,7
3,5,2,0

result: 22
"""


def find_max_sum_in_matrix(matrix: list[list[int]]) -> int:
    # Stage 1: Initialization
    rows: int = len(matrix)
    cols: int = len(matrix[0])
    row_sums: list[int] = [0] * rows
    col_sums: list[int] = [0] * cols

    # Stage 2: Preprocess the row and col sums with the prefix lists
    for row in range(rows):
        for col in range(cols):
            row_sums[row] += matrix[row][col]
            col_sums[col] += matrix[row][col]

    # Stage 3: Search for the response using the calculated sums
    max_sum: int = 0
    for row in range(rows):
        for col in range(cols):
            row_sum: int = row_sums[row] - matrix[row][col]
            col_sum: int = col_sums[col] - matrix[row][col]
            max_sum = max(max_sum, row_sum + col_sum)

    return max_sum


if __name__ == "__main__":
    matrix: list[list[int]] = [[1, 5, 1, -1], [3, 1, 2, 7], [3, 5, 2, 0]]

    response: int = find_max_sum_in_matrix(matrix)
    print(f"Matrix: {matrix} -> Max sum: {response}")
```

Chapter 9
Stack



9.1 Monotonic Stack

Complexity

Time: O(n) **Memory:** O(n)

Green Flags -- When to Use Monotonic Stack

- If we need to search for the first greater/smaller element, we can use a monotonic stack

Approach

Keep the stack strictly increasing or strictly decreasing at all times; whenever a new element would break that order, pop everything it beats and resolve each popped element's answer using the new element (e.g. "next greater element"), then push the new element (or its index). Because every element is pushed and popped at most once, the whole scan stays linear despite the nested-looking while loop.

Pattern Template

```
"""
Pattern: monotonic stack

The stack is kept strictly increasing or strictly decreasing; when a
new value breaks that order, everything it "beats" gets its answer
resolved. Indices are pushed instead of values so distance can be
measured, and values can still be recovered via nums[index].

Time: O(n) -- each index is pushed and popped at most once
```

```
    Memory: O(n) -- for the stack and the result array
    """


def next_greater(nums: list[int]) -> list[int]:
    # ============================================================================
    # Stage 1: -1 means "no greater element found yet"
    # ============================================================================
    result = [-1] * len(nums)
    stack: list[int] = []

    # ============================================================================
    # Stage 2: Decide the scan direction (left-to-right vs right-to-left)
    # based on what the stack needs to already contain at each index
    # ============================================================================
    for i in ...:
        # ========================================================================
        # Stage 3: While the current value breaks the monotonic order, pop
        # and resolve the popped index's answer, then push the current index
        # ========================================================================
        ...

    return result
```

Worked Example

```
"""
Given a list of numbers nums, we should return a new list where
the index of original contains the first element greater than its value.
Otherwise return -1.

e.g.
nums = [2,5,1,3,0,4]
output = [5,-1,3,4,4,-1]
"""


def find_greaters(nums: list[int]) -> list[int]:
    result: list[int] = [-1] * len(nums)
    stack: list[int] = []

    i: int = len(result) - 1
    while i >= 0:
        if len(stack) == 0:
            stack.append(nums[i])
            i -= 1
        elif nums[i] < stack[-1]:
            result[i] = stack[-1]
            stack.append(nums[i])
            i -= 1
        else:
            stack.pop()

    return result


def find_greaters_optimized(nums: list[int]) -> list[int]:
    n = len(nums)
    stack = []
    result = [-1] * n

    for i in range(n - 1, -1, -1):
        while stack and nums[i] >= stack[-1]:
            stack.pop()

        if stack:
            result[i] = stack[-1]
```

```
            stack.append(nums[i])

    return result


if __name__ == "__main__":
    nums: list[int] = [2, 5, 1, 3, 0, 4]
    print(f"Numbers: {nums}")
    result: list[int] = find_greaters(nums)
    result_optimized: list[int] = find_greaters_optimized(nums)
    print(f"Result: {result}")
    print(f"Result optimized: {result_optimized}")
```

9.2 Pseudo Stack

Complexity

Time: O(n) **Memory:** O(1)

Green Flags -- When to Use Pseudo Stack

- If the problems seems as for stacks, but those stacks can be changed to a counter or various counter
- Inside the stack we push only the same elements
- The size of the stack is what matters, not the actual values

Approach

When the only thing a stack would ever hold is many copies of the same symbol, its contents carry no information beyond their count – so replace the stack with a single counter that increments on "open" and decrements on "close", and treat an attempted decrement below zero as an immediate invalid case.

Pattern Template

```
"""
Pattern: pseudo stack (counter instead of a real stack)

When the only thing a stack would ever hold is copies of one kind of
symbol, its contents carry no information beyond their count -- so a
single counter replaces the stack entirely.

Time: O(n)
Memory: O(1)
"""


def is_valid(text: str) -> bool:
    # ============================================================================
    # Stage 1: depth stands in for "how many unmatched openings so far"
    # ============================================================================
    depth = 0

    # ============================================================================
    # Stage 2: An opening symbol increases the depth, a closing symbol
    # decreases it -- and a closing symbol seen at depth 0 means there is
    # nothing left to match, so the input is already invalid
```

```
    # ================================================================================
    for char in text:
        ...

    return depth == 0
```

Worked Example

```
"""
Given a list of same symbols, we need to check if every opening symbol
has its closing symbol.

e.g.
symbols = ["(",")","(","(",")",")"]
output = True

symbols = ["(","(",")"]
output = False

This approach really does not need the stack, we can just use a counter kek.
"""


def is_valid(symbols: list[str]) -> bool:
    counter: int = 0
    for symbol in symbols:
        if symbol == "(":
            counter += 1
        else:
            counter -= 1

        if counter < 0:
            return False

    return counter == 0


if __name__ == "__main__":
    symbols: list[str] = ["(", ")", "(", "(", ")", ")"]
    bad_symbols: list[str] = ["(", "(", ")"]

    print(f"Symbols: {symbols} -> {is_valid(symbols)}")
    print(f"Bad symbols: {bad_symbols} -> {is_valid(bad_symbols)}")
```

9.3 Stack of Intermediate Results

Complexity

Time: O(n) **Memory:** O(n)

Green Flags -- When to Use Stack of Intermediate Results

- We should work with bracket expressions or soime similar structures
- We should calculate the result of the expression
- Stack is used for an intermediate results

Approach

Use an actual stack (not just a counter) whenever more than one kind of pending result can be open simultaneously, because the algorithm must remember not just how many things are open, but specifically which one has to be closed next – exactly what a stack's LIFO order guarantees.

Pattern Template

```
"""
Pattern: stack of intermediate results

A real stack (not just a counter) is needed whenever more than one
kind of pending result can be open at once, because the algorithm must
remember *which* one is waiting to be closed next, not just how many.

Stacks are LIFO (Last In, First Out): push(x) adds on top, pop()
removes and returns the top, peek() reads the top without removing it.

Time: O(n)
Memory: O(n) -- worst case every symbol is pushed (e.g. all openings)
"""


def is_valid(symbols: list[str]) -> bool:
    # ============================================================================
    # Stage 1: The stack holds whatever "intermediate result" is still
    # needed later -- here, unmatched opening symbols
    # ============================================================================
    stack: list[str] = []

    # ============================================================================
    # Stage 2: Push openings; on a closing symbol, pop and check it
    # matches what was expected
    # ============================================================================
    for symbol in symbols:
        ...

    return len(stack) == 0
```

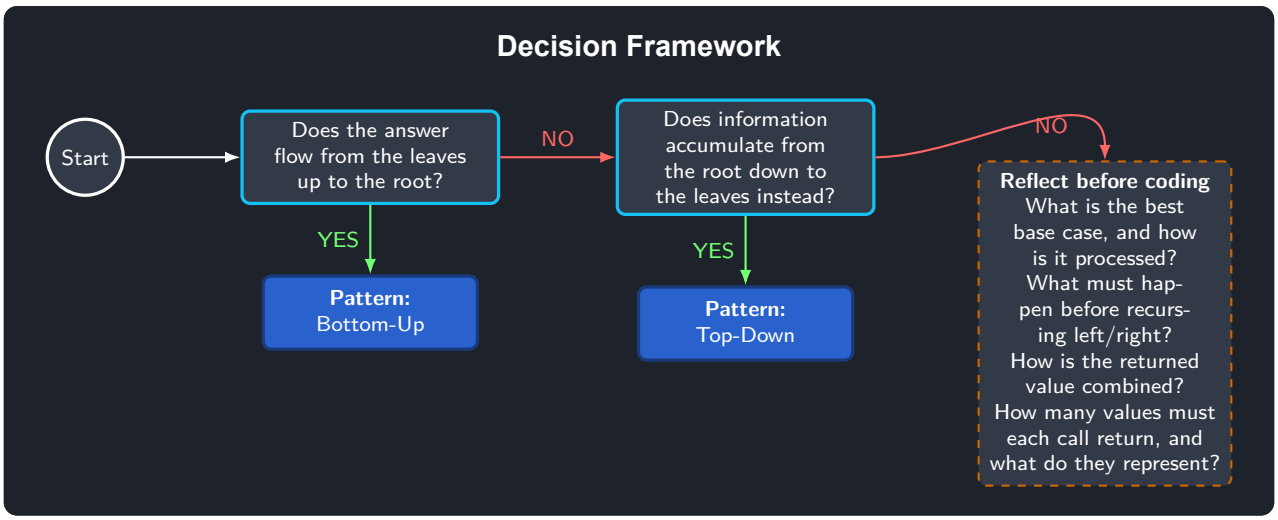
Worked Example

```
"""
We are given with a list of symbols, we need to check if every opening symbol
has its closing symbol.

e.g.
symbols = ["(","(", "<", "[","{","}","]",">"]
output = True

symbols = ["(","(",")"]
output = False

This is a stack problem, so we need to use a stack to keep track of the opening symbols.
"""


def is_good(symbols: list[str]) -> bool:
    stack: list[str] = []

    patterns: dict[str, str] = {"(": ")", "[": "]", "{": "}", "<": ">"}

    for symbol in symbols:
        if symbol in patterns:
            # CHAR is an opening symbol, so we add it to the stack
```

```
                stack.append(symbol)
            elif len(stack) == 0:
                # CHAR is a closing symbol, but without opening one, so finish that
                return False
            else:
                # CHAR is closing one, but we have elements in the stack, so we check
                top = stack.pop()
                if symbol != patterns[top]:
                    return False

        return len(stack) == 0


if __name__ == "__main__":
    symbols: list[str] = ["(", ")", "<", "[", "{", "}", "]", ">"]
    bad_symbols: list[str] = ["(", "(", ")"]

    print(f"Symbols: {symbols} -> {is_good(symbols)}")
    print(f"Bad symbols: {bad_symbols} -> {is_good(bad_symbols)}")
```

Chapter 10

Trees



10.1 Bottom-Up

Complexity

Time: O(n) **Memory:** O(n)

Foundations

```
def count_nodes(node: TreeNode) -> int:
    if node is None:
        return ... # Base case: what should be returned for the empty three?

    left = count_nodes(node.left) # Recursive call to the left subtree
    right = count_nodes(node.right) # Recursive call to the right subtree

    return ... # Form the response in here to pass to the parent
```

Typical error while paying with trees

- Do extra check while processing the info in the base case

```
# BAD
def count_nodes(node: TreeNode) -> int:
    if node.left is None and node.right is None:
        return 1
```

```
...
```

- Forget to add the base case.

> ![IMPORTTANT] If you are working with trees (recursion), you should ALWAYS ADD A BASE CASE IN ORDER TO CHECK FOR THE VOID LEAF.

Green Flags -- When to Use Bottom-Up

- We need to accumulate the response from leaves to the root.
- The response on the parent depends on the responses from the children.

Approach

Recurse into both children first, and only then combine their two results into the value this node hands back to its own parent. Get the base case (an empty/`None` node) right first, since it defines what an "empty contribution" looks like, and every other computation is built on top of it.

Pattern Template

```python
from patterns.tree.lib import TreeNode

"""
Pattern bottom up

Time: O(n)
Memory: O(n)
"""


def find_diameter(node: TreeNode) -> int:
    def dfs(node: TreeNode) -> int:
        # ============================================================================
        # Stage 1: Base case, what to do for void leaf?
        # ============================================================================
        if node is None:
            return ...

        # ============================================================================
        # Stage 2: Recursive call to the leaves
        # ============================================================================
        ...

        # ============================================================================
        # Stage 3: Updating the result, knowing the response from the subtrees
        # Base problem is to understand how to form the result
        # ============================================================================
        ...

        # ============================================================================
        # Stage 4: Return the result to the parent
        # And what we should return to the parent
        # ============================================================================
        return ...

    return dfs(node)
```

Worked Example

```
"""
Given the tree root, you should find its diameter:
the diameter is the length of the longest path between any two nodes in a tree.

e.g.

                1
            /
            2
        / \\
        3 4
        \\ \\
        5 6

5->3->2->4->6 = 4
solution= 4
"""

from patterns.tree.lib import TreeNode


def find_tree_diameter(node: TreeNode) -> int:
    diameter: int = 0

    # Lets do a recursion

    def dfs(node: TreeNode) -> int:
        # Base case ALWAYS
        if node is None:
            return 0

        # Base case, we add how many nodes we have in the branches
        left = dfs(node.left)
        right = dfs(node.right)

        # Obtain the max diameter, this is the key response here
        nonlocal diameter
        diameter = max(diameter, left + right)

        # Now we should select the maximum nodes in the subtree
        return max(left, right) + 1

    dfs(node)
    return diameter


if __name__ == "__main__":
    root: TreeNode = TreeNode(
        1,
        TreeNode(
            2,
            TreeNode(3, TreeNode(5, None, None), None),
            TreeNode(4, None, TreeNode(6, None, None)),
        ),
        None,
    )

    print("Diameter:", find_tree_diameter(root))
```

10.2 Top-Down

Complexity

Time: O(n) **Memory:** O(h)

Green Flags -- When to Use Top-Down

- We need to pass extra values from the root to the leaves to form the response.

Approach

Carry information downward as an extra function argument that gets updated on the way from the root towards the leaves (a running sum, a running depth, an allowed range), and let each node use that accumulated context to decide or record part of the final answer – the opposite data flow from the bottom-up pattern.

Pattern Template

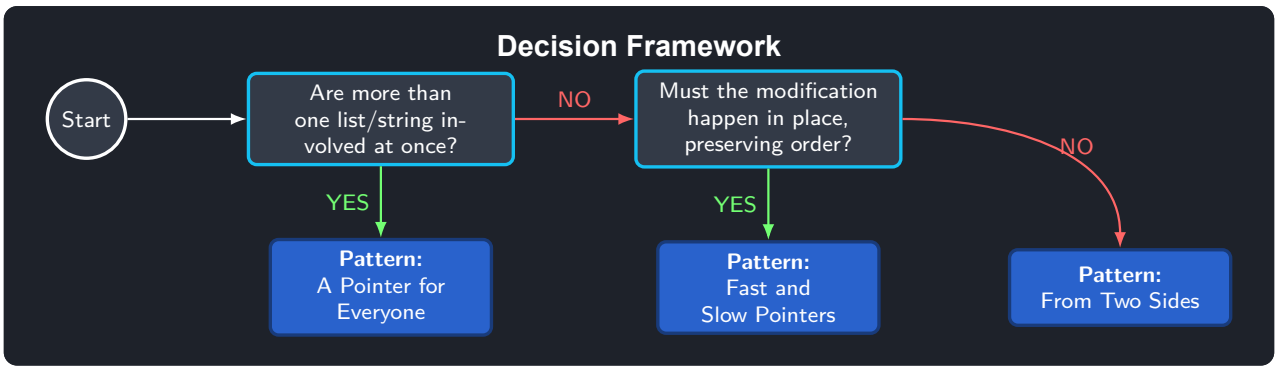
```python
"""
Up to bottom recursion tree pattern

Time: O(n)
Memory: O(h)
"""
from patterns.tree.lib import TreeNode

def sum_exists(root: TreeNode, target_sum: int = 0)-> bool:
    def dfs(node: TreeNode, ...) -> bool:
        # ============================================================================
        # Stage 1: Base case, what to do for void leaf?
        # ============================================================================
        if node is None:
            ...

        # ============================================================================
        # Stage 2: Recursive call to the leaves
        # ============================================================================
        ...

        # ============================================================================
        # Stage 3: Updating the result, knowing the response from the subtrees
        # Base problem is to understand how to form the result
        # ============================================================================
        return ...

    # Base problem is to understand what to pass to the children and how the recursion should work
    return dfs(root, 0)
```

Worked Example

```python
"""
Given a root for a binary tree. Yout should return all the values
by levels, starting from the root and going to the leaves.

e.g.

root = [3,9,20,null, null, 15, 7]

    3
   / \\
  9 20
    / \\
   15 7

response0 [[3], [9,20], [15,7]]
"""

```

```python
from patterns.tree.lib import TreeNode, build_tree


def values_by_level(node: TreeNode) -> list[list[int]]:
    result: dict[int, list[int]] = {}

    def dfs(node: TreeNode, level: int) -> bool:
        if node is None:
            return False

        # Add the value to the list
        nonlocal result
        if node.value is not None:
            result[level] = result.get(level, []) + [node.value]

        # Recursive call to the leaves
        dfs(node.left, level + 1)
        dfs(node.right, level + 1)

    dfs(node, 0)
    return list(result.values())


if __name__ == "__main__":
    tree_list: list[int | None] = [3, 9, 20, None, None, 15, 7]
    tree: TreeNode = build_tree(tree_list)

    print(f"Tree {tree_list} has values: {values_by_level(tree)}")
```

Chapter 11
Two Pointers



11.1 Fast and Slow Pointers

Complexity

Time: O(n) **Memory:** O(1)

Green Flags -- When to Use Fast and Slow Pointers

- We need in-place modifications for lists or strings
- We need to remain with the same order of the elements

Approach

Use two pointers moving through the same list or array at different speeds (or one pointer writing while another reads) to modify the structure in place, in a single linear pass, while preserving the relative order of the remaining elements – the classic use case is partitioning or removing elements without allocating a second array.

Pattern Template

```python
# Time: O(n)
# Memory: O(1)


def move_all_m_to_end_pattern(nums: list, target: int) -> list[int]:
    # ============================================================================
    # 1) Slow and fast pointers initializations
    # ============================================================================
```

```
    slow = 0
    fast = 0

    # ================================================================================
    # 2) While loop while we check all the conditions
    # ================================================================================
    while fast < len(nums):
        # ============================================================================
        # 3) Pointer logic movement: IS THE KEY TO THE SOLUTION, UNDERSTAND WHEN
        # TO MOVE THEM, is this is understood, the problem is solved automatically
        # ============================================================================
        ...
        # The fast pointer moves always but the slow only in some cases.
        # Since the fast pointer moves always, therefore the main problem
        # is to understand when to move the slow pointer
```

Worked Example

```
"""
Given a list with n numbers, we need to move all the m numbers to the end of the list.
The approach is to use two pointers, one for the fast pass and another for the slow pass.

It should be done inplace, aka without creating a new list basically.
"""

import os

DEFAULT_LIST: list[int] = [1, 0, 1, 1, 5, 42, 7] # Solution: [0,5,42,7,1,1,1]
DEFAULT_NUMBER: int = 1


def read_input() -> tuple[list[int], int]:
    try:
        with open(os.path.join(os.path.dirname(__file__), "input.txt")) as f:
            inputs = list(map(int, f.readline().strip("\n").split(",") or DEFAULT_LIST))
            number = int(f.readline().strip("\n") or DEFAULT_NUMBER)
            return inputs, number

    except FileNotFoundError:
        return DEFAULT_LIST, DEFAULT_NUMBER


def move_all_ones_to_end(
    nums: list[int], loockup_num: int = DEFAULT_NUMBER
) -> list[int]:
    """Given the array, it moves all the ones to the end of the list
    using the fast and slow pointers pattern
    """
    p_fast: int = 0
    p_slow: int = 0

    while p_slow < len(nums) and p_fast < len(nums):
        if nums[p_fast] == loockup_num:
            p_fast += 1
            continue

        elif nums[p_slow] == loockup_num and nums[p_fast] != loockup_num:
            # Use just pointers to move the ones to the end
            nums[p_slow], nums[p_fast] = nums[p_fast], nums[p_slow]

        # Edge case when the slow and fast pointers start inside a bad number
        p_slow += 1
        p_fast += 1


def move_all_ones_to_end_optimized(
    nums: list[int], loockup_num: int = DEFAULT_NUMBER
```

```python
) -> list[int]:
    """Given the array, it moves all the ones to the end of the list
    using the fast and slow pointers pattern
    """
    p_fast: int = 0
    p_slow: int = 0

    while p_fast < len(nums):
        if nums[p_fast] != loockup_num:
            nums[p_slow], nums[p_fast] = nums[p_fast], nums[p_slow]
            p_slow += 1

        # The fast pointer should be moving always
        p_fast += 1


if __name__ == "__main__":
    inputs, loockup_num = read_input()
    print(f"Array: {inputs} | Number: {loockup_num}")
    move_all_ones_to_end(nums=inputs, loockup_num=loockup_num)
    print(f"Solution: {inputs} \n")

    inputs, loockup_num = read_input()
    move_all_ones_to_end_optimized(nums=inputs, loockup_num=loockup_num)
    print(f"Solution: {inputs}")
```

11.2 From Two Sides

Complexity

Time: O(n) **Memory:** O(1)

Green Flags -- When to Use From Two Sides

- The response it forms by doing the list smaller by both sides at the same time.

Approach

Start one pointer at each end of a sorted (or otherwise structured) sequence and move them towards each other based on a comparison at each step, discarding the side that cannot possibly be part of the answer – this halves the remaining search space on every non-matching step, with no nested loops needed.

Pattern Template

```python
# Time: O(n)
# Memory: O(1)


def two_pointer_pattern(nums: list, target: int) -> list[int]:
    # ================================================================================
    # 1) L & R initializations
    # ================================================================================
    l = 0
    r = len(nums) - 1

    # ================================================================================
    # 2) While loop while we check all the conditions
    # ================================================================================
    while l < r:
```

```
        # ================================================================================
        # 3) Pointer logic movement: IS THE KEY TO THE SOLUTION, UNDERSTAND WHEN
        # TO MOVE THEM, is this is understood, the problem is solved automatically
        # ================================================================================
        ...
        # <The L and R pointers are moved here>
```

Worked Example

```
"""
Given world s. You need to find if this world is a palindrome or not.
yes: 12321 -> 12321
no: 123 -> 321
"""


def is_palindrome(s: str) -> bool:
    """The easiest way here is to count the chars in the string and use the dictionary counting
    approach, since palindromes should have the even number of chars or one odd and the other even.
    """

    l: int = 0
    r: int = len(s) - 1

    while l < r:
        # Here we should check the opposite chars, so we need to move the counters
        # at the same time
        if s[l] != s[r]:
            return False
        l += 1
        r -= 1
    return True


if __name__ == "__main__":
    print(f"FALSE: {is_palindrome("123")}")
    print(f"TRUE: {is_palindrome("12321")}")
    print(f"FALSE: {is_palindrome("12")}")
    print(f"TRUE: {is_palindrome("1")}")
    print(f"FALSE: {is_palindrome("hola")}")
    print(f"TRUE: {is_palindrome("ABBA")}")
```

11.3 A Pointer for Everyone

Complexity

Time: O(a+b) **Memory:** O(min(a,b))

Green Flags -- When to Use A Pointer for Everyone

- Given various lists or strings
- We need search for union or interesection etc for two sorted lists
- Basically if you see two lists or two strings, you should check for this pattern

Approach

When two or more separate sequences must be reconciled with each other (merged, intersected, compared), keep one independent pointer per sequence and advance whichever pointer is “behind”

at each step, based on a per-element comparison – since each pointer only ever moves forward, the whole reconciliation stays linear in the combined input size.

Pattern Template

```python
# Time: O(a+b)
# Memory: O(min(a,b))


def pointer_for_everyone(a: list, b: list) -> list:
    # ================================================================================
    # 1) Pointers initializations
    # ================================================================================
    p1: int = 0
    p2: int = 0
    result: list = []

    # ================================================================================
    # 2) While loop while we check all the conditions
    # ================================================================================
    while p1 < len(a) and p2 < len(b):
        # ============================================================================
        # 3) Pointer logic movement: IS THE KEY TO THE SOLUTION, UNDERSTAND WHEN
        # TO MOVE THEM, is this is understood, the problem is solved automatically
        # ============================================================================
        ...
        # <The L and R pointers are moved here>

    return result
```

Worked Example

```python
"""
Given two lists: [0,2,4] and [1,2,2,8] you should return all the common elements.

Response here: [2]
"""

import os

LIST_A: list[int] = [0, 2, 4, 5, 8]
LIST_B: list[int] = [1, 2, 3, 4, 8, 8, 8]


def read_input() -> tuple[list, int]:
    """Reads the input, first the array and then the target, it returns the tuple of the array in
    an int format and the target in an int format"""
    try:
        with open(os.path.join(os.path.dirname(__file__), "input.txt")) as f:
            array_a = list(map(int, f.readline().strip("\n").split(",") or LIST_A))
            array_b = list(map(int, f.readline().strip("\n").split(",") or LIST_B))
            return array_a, array_b
    except FileNotFoundError:
        return LIST_A, LIST_B


def common_elements(array_a: list, array_b: list) -> list[int]:
    """The basic approach is also to have two pointers on the start, move the smaller one,
    if they are equal, then we mark them as found and move both one step.

    The current approach only works if the list are sorted"""
    common_elements: list[int] = []
    p1: int = 0
    p2: int = 0

```

```
    while p1 < len(array_a) and p2 < len(array_b):
        if array_a[p1] < array_b[p2]:
            p1 += 1
        elif array_b[p2] < array_a[p1]:
            p2 += 1
        else:
            common_elements.append(array_a[p1])
            p1 += 1
            p2 += 1

    return common_elements


if "__main__" == __name__:
    array_a, array_b = read_input()
    print(f"Array A: {array_a} | Array B: {array_b}")
    solution = common_elements(array_a, array_b)
    print(f"Solution: {solution}")
```